

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a deep blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene, digital aesthetic.

DL for ENG

## LECTURE 3.1

# What is Artificial Intelligence?

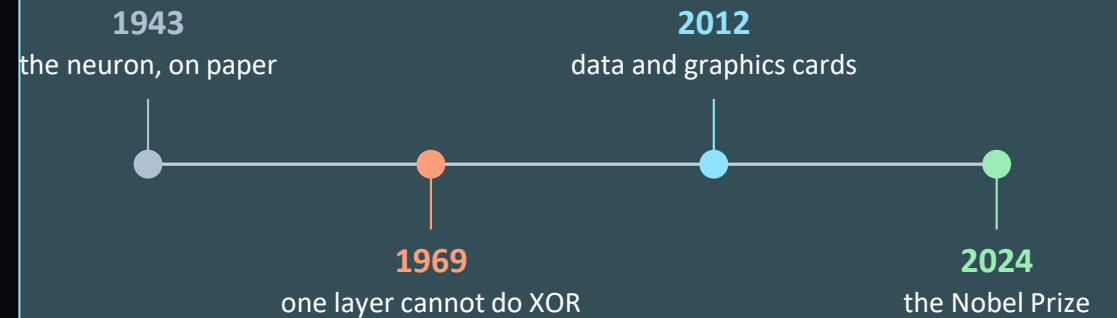
**Remus Teodorescu**

*With support from Research Assistant Noman Khan -  
nomank@energy.aau.dk*

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

### DEFINITIONS · HOW IT LEARNS · JUDGEMENT



*Three words that are used as if they meant the same thing, and do not: artificial intelligence, machine learning, deep learning. Today is what each one is, where the training signal comes from in the three kinds of learning, and the judgement an engineer needs most, which is knowing when a trained network is the wrong tool and a solver is the right one.*

# Learning Objectives

## SECTIONS, IN ORDER

- 1 Definitions** — artificial intelligence, machine learning, the neural network, deep learning, the language model
- 2 How it learns** — supervised, unsupervised, reinforcement
- 3 Judgement** — the modelling spectrum, where it fits, when not to use it, how much data

*No mathematics today beyond one energy function and one line defining optimisation. The machinery starts in the next lecture.*

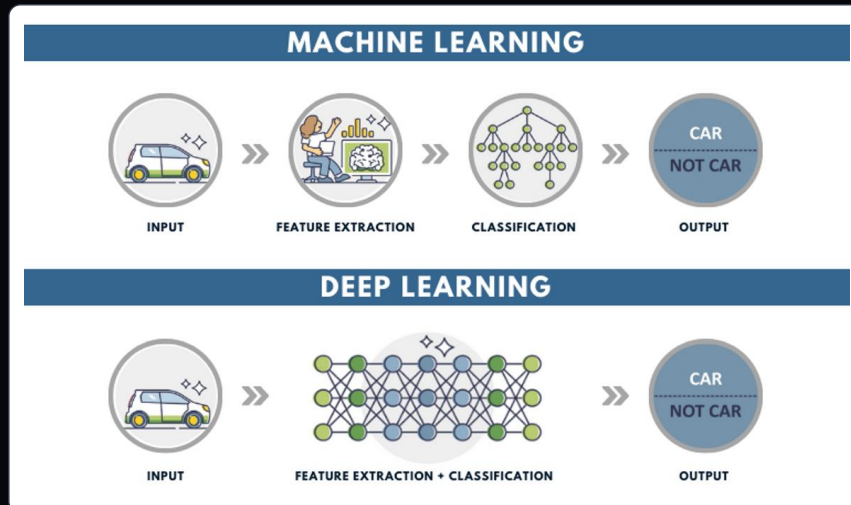
## READ, AND WHAT EACH ANSWERS

- |                          |                                                 |
|--------------------------|-------------------------------------------------|
| <b>UDL ch. 1</b>         | what do the four words actually mean?           |
| <b>UDL ch. 1.1 – 1.3</b> | where does the training signal come from?       |
| <b>UDL ch. 1.4</b>       | when is a network the right tool, and when not? |

# Artificial Intelligence

**Artificial intelligence** — the whole ambition of making a machine do what would take intelligence in a person: search, planning, logic, robotics and language. Most of it involves no learning at all. Machine learning is the part that fits models to data, and deep learning is the part of that which stacks many layers.

## WHO EXTRACTS THE FEATURES



## FOUR AREAS, ONE INSIDE THE NEXT

### ARTIFICIAL INTELLIGENCE

search · planning · reasoning · robotics

### MACHINE LEARNING

regression · k-means · PCA · SVM · trees

### NEURAL NETWORKS

perceptron · MLP · Hopfield · Boltzmann

### DEEP LEARNING

CNN · RNN · GNN · transformers · PINNs

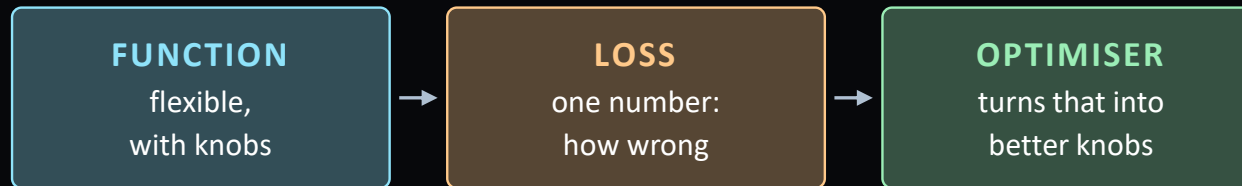
## THE ONE TO REMEMBER

A Kalman filter is AI by any 1970s definition, and still the best tool for state estimation in most control rooms, where it is still the baseline a neural method has to beat, and often will not.

# Machine Learning

**Machine learning** — fitting a model to data so that it decides or predicts without being given the rule. It takes three ingredients: a function with adjustable parameters, a loss that says in one number how wrong it is, and an optimiser that makes that number smaller. Linear regression counts, and so does a billion-parameter network.

## THE THREE INGREDIENTS



$$\hat{\theta} = \operatorname{argmin}_{\theta} \mathcal{L}(\theta)$$

## THE SAME THREE, LECTURE BY LECTURE

|             |                                    |
|-------------|------------------------------------|
| <b>L4</b>   | the function gets deep             |
| <b>L5</b>   | the function gets structure        |
| <b>L6.1</b> | the loss gets built properly       |
| <b>L7</b>   | the loss gets physics in it        |
| <b>L12</b>  | the function is a whole power grid |

## WHAT ACTUALLY CHANGES IN PART 2

Only the middle one. The function stays a network and the optimiser stays gradient descent. Physics-informed learning is a statement about the loss, and nothing else.

### THE THREE INGREDIENTS IN PYTORCH

```

model = nn.Linear(1, 1)           # the function: two knobs
loss = ((model(x) - y) ** 2).mean() # the loss: how wrong it is
opt = torch.optim.SGD(model.parameters(), lr=0.1) # the optimiser
opt.zero_grad(); loss.backward(); opt.step()      # better knobs
  
```

# Neural Network

**Neural network** — a function built from many small units, each taking a weighted sum of its inputs and passing it through a nonlinearity. The weights are what training adjusts. It was written down in 1943 as a model of a biological neuron, and the borrowed word has misled people ever since.

## FROM THE FIRST NEURON TO THE NOBEL PRIZE

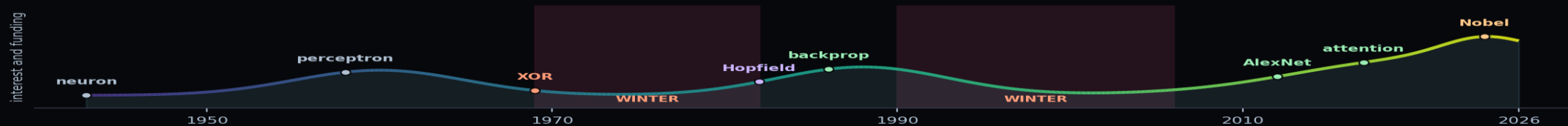
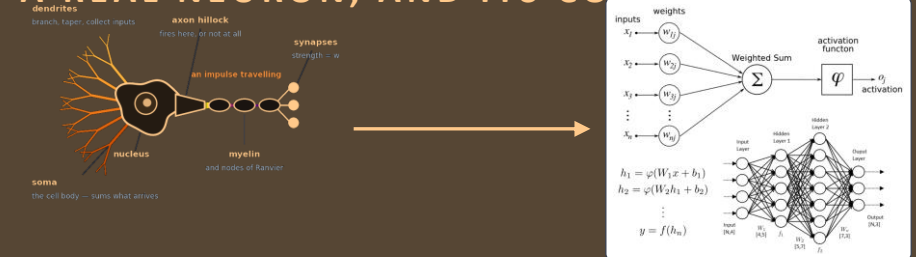
- **1943 • McCulloch and Pitts** — a threshold on a weighted sum
- **1958 • Rosenblatt** — the perceptron, built in hardware
- **1969 • Minsky and Papert** — one layer cannot do XOR
- 1986 • backpropagation trains many layers, ending the winter
- **2024 • Hopfield and Hinton** — the Nobel Prize in Physics

## HOPFIELD'S ENERGY

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

Neural networks began in statistical physics: the Hopfield network (1982) and Hinton's Boltzmann machine (1985) are spin-glass energy landscapes. This course takes them back to engineering; the prize that recognised the origin is two slides on.

## A REAL NEURON, AND ITS COPY



# Deep Learning

**Deep learning** — machine learning with a function made of many layers, each one feeding the next, fitted end to end by gradient descent. What the layers buy is that nobody designs the features: the early layers learn what to measure and the later ones learn what to do with it. The method was correct in 1986 and unusable until the data and the arithmetic arrived.

## WHY 2012 AND NOT 1992

- **1986 to 2012** — the mathematics barely changed, and SVMs won
- **2012 · AlexNet** — 1.2 million labels, two graphics cards
- **the lesson** — too little data makes a correct method look wrong

## ALEXNET, 2012 · KRIZHEVSKY, SUTSKEVER AND HINTON



*Eight layers, 60M parameters. 15.3 % ImageNet error, against 26.2 %.*

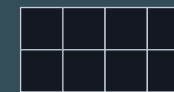
## WHAT ARRIVED, AND WHEN

|                       |                                   |
|-----------------------|-----------------------------------|
| <b>the idea</b>       | 1986 — and then it waited         |
| <b>the data</b>       | ImageNet, 2009, 14 million images |
| <b>the arithmetic</b> | GPUs, and someone willing to try  |

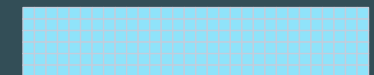
## THE ONE TO REMEMBER

Too little data or compute makes a correct method look wrong.

## GPU VS CPU



CPU · 8 cores, in sequence



GPU · thousands, all at once

A matrix product is one operation on thousands of numbers. CUDA (2006) let 3D-game cores do any arithmetic.

# Large Language Model

**Large language model** — a network trained on an enormous quantity of text to predict the next word, then trained further to follow instructions. What it computes at each step is a distribution over the next piece of text, and nothing else. It is strong where the answer has been written down, and unreliable where it must be derived.

## WHAT THE BENCHMARKS ACTUALLY ASK

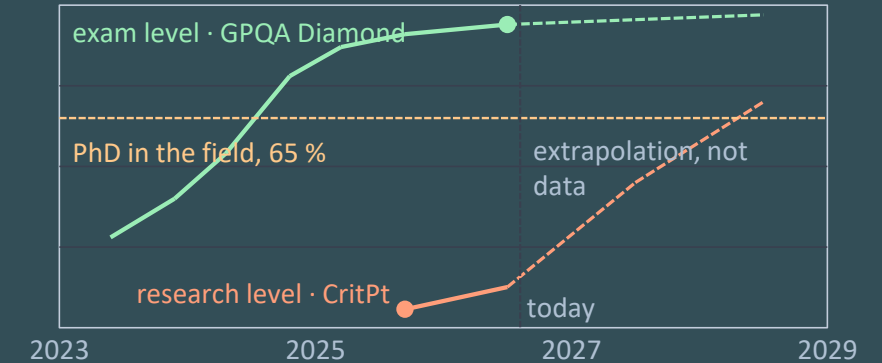
- **exam level** — a PhD scores 65 % on GPQA, models passed it
- **two meanings of research** — find one, or derive one
- in engineering the check is physical

## WHAT 'RESEARCH' MEANS ON A LEADERBOARD

| # | model           | maker     | score | what it blends        |
|---|-----------------|-----------|-------|-----------------------|
| 1 | GPT-6 Astra     | OpenAI    | 93.8  | GPQA · HLE ·          |
| 2 | GPT-5.6 Sol     | OpenAI    | 93.4  | Browse Science        |
| 3 | GPT-5.6 Terra   | OpenAI    | 90.2  | DeepSearch-QA         |
| 9 | Claude Mythos 5 | Anthropic | 82.2  | GAIA                  |
|   |                 |           |       | benchlm.ai, 76 models |

Every benchmark in the blend has an answer that exists; CritPt's has to be derived. Knowing what is known is 93.8; working out what is not is 5.7.

## EXAMS PASSED. RESEARCH NEXT.



## THE CONCLUSION

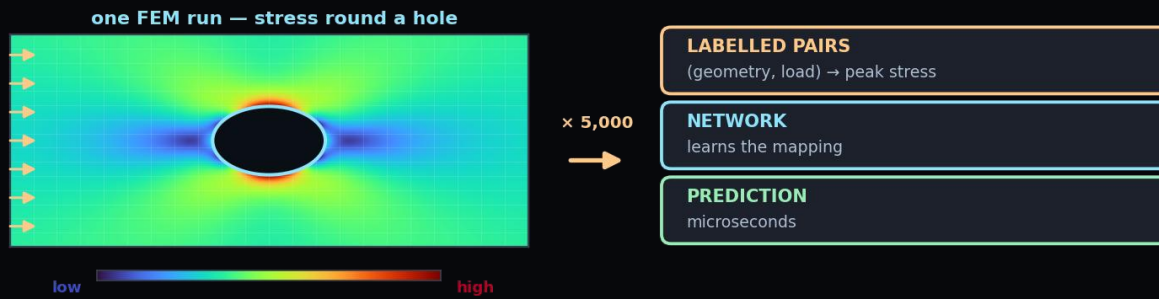
Language models accelerate your research; they do not replace you in it. A language model is fluent about physics; a physics-informed network is constrained by it. The PINN on our oscillator is wrong by 0.59 mm and knows which equation it is solving. Use the first to build the second.

# Supervised Learning

**Supervised learning** — learning from inputs whose answers are already known: every example carries the output it should produce, and the answers are the supervision. It is the setting of almost everything in Part 1. Its cost is that somebody had to produce those answers, by measuring or by labelling, before any training could start.

## WHERE THE ANSWERS COME FROM

- **the answers are the supervision** — somebody made them
- **in engineering** — a simulation is a label factory, run once
- **Ex\_03 is supervised** — time in, displacement out



the answers already exist

## IN THIS COURSE

|              |                              |
|--------------|------------------------------|
| <b>Ex_03</b> | fit a curve, three ways      |
| <b>Ex_04</b> | classify, and fail at XOR    |
| <b>Ex_05</b> | images, and a six-bus grid   |
| <b>L11.1</b> | steering angle from a camera |

**TALK · 2 MIN** Where would the labelled answers for your own project come from, and who would have to produce them? Then, in the table on the right, which of those four is the one where labels are cheapest, and why?

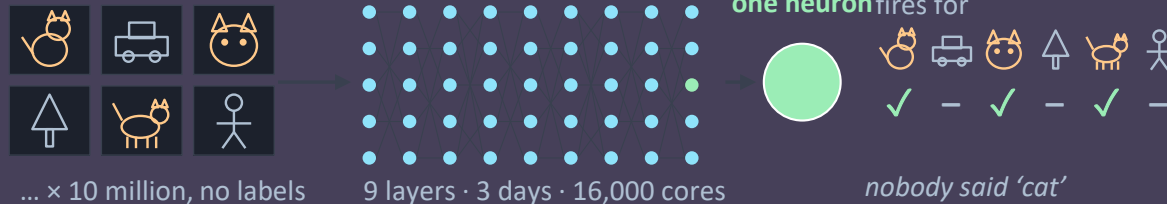
# Unsupervised Learning

**Unsupervised learning** — learning from data with no answers attached, to find the structure that is already in it: clusters, axes of variation, or departures from normal. Its other face is learning features from unlabelled data, which is how a model is pre-trained before a handful of labels are used. Nothing guarantees the structure it finds is the one you wanted.

## STRUCTURE, AND FEATURES WITHOUT LABELS

- **the structure already there** — clusters, axes, anomalies
- **in engineering** — learn normal from logs, then flag departures
- **the catch** — the structure found need not be the one you wanted

### GOOGLE BRAIN 2012 · ONE NEURON FIRES FOR CATS



## WHAT IS IN SCOPE

|                          |                                 |
|--------------------------|---------------------------------|
| <b>PCA</b>               | yes — mentioned in L6.2         |
| <b>clustering</b>        | yes — briefly                   |
| <b>anomaly detection</b> | yes — the engineering case      |
| <b>generative models</b> | out of scope — UDL ch. 14 to 18 |

## LEARNING FEATURES WITHOUT LABELS

|                           |                                                          |
|---------------------------|----------------------------------------------------------|
| <b>Hinton, 2006</b>       | deep belief nets: unlabelled digits, a few labels on top |
| <b>Google Brain, 2012</b> | a 'cat' neuron from 10 million unlabelled frames         |
| <b>language models</b>    | pre-trained the same way, on unlabelled text             |

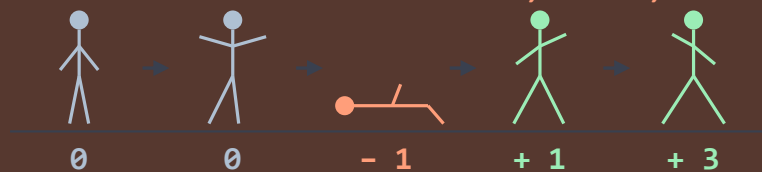
# Reinforcement Learning

**Reinforcement learning** — learning to act from consequences rather than answers. The model acts, the world responds with a reward that usually arrives late, and the model adjusts to make high-reward actions more likely. Supervised learning is told the answer, unsupervised learning is told nothing, and this is told how good its answer was, afterwards.

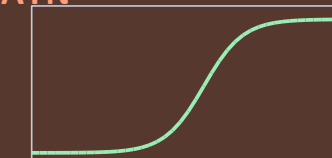
## ACT, OBSERVE, ADJUST

- **no answers, only consequences** — and they arrive late
- the reward is the design problem, not a detail
- **the cost** — millions of trials, so it lives in a simulator

### LEARNING TO WALK · TRY, FALL, TRY AGAIN



attempt → consequence → adjust. No labels, no dataset: the floor is the teacher.



reward against attempts

## THE VOCABULARY, TRANSLATED

|                |                           |
|----------------|---------------------------|
| <b>state</b>   | what the plant is doing   |
| <b>action</b>  | the control input         |
| <b>policy</b>  | the controller            |
| <b>reward</b>  | minus the cost function   |
| <b>episode</b> | one run of the experiment |

## WHERE IT IS TAUGHT

L6.2 teaches the fundamentals and the role of reinforcement in post-training. L11 runs it on a JetRacer, where a bad policy costs you a physical crash rather than a number in a table.

# The Modelling Spectrum

**The modelling spectrum** — the line running from a model built entirely from physics to one built entirely from data, on which every model sits somewhere. Pure physics extrapolates and is slow. Pure data is fast and silent outside the range it was fitted on. The question is never physics or data, but how much of each.

## ONE LINE, NOT TWO CAMPS

- **pure physics** — extrapolates, and is slow
- **pure data** — fast, and silent outside its range
- every model you have built already sits somewhere on it



## THE LINE, END TO END



|              |                                        |
|--------------|----------------------------------------|
| pure physics | no data, extrapolates, slow            |
| hybrid       | physics written into the loss — Part 2 |
| pure data    | fast, silent outside its range         |

**TALK · 2 MIN** Take a model you have built yourself. Where on this line does it sit, and what would move it one step towards physics? Then, on the bar above, which of the five markers is closest to it?

# Where It Fits

**Where it fits** — the four situations in which a trained network earns its place: interpolating where the data is dense, working in many dimensions where nobody can write the correlations down, evaluating in microseconds once trained, and recovering a parameter nobody can measure.

## FOUR SITUATIONS, ONE SHAPE

- **interpolation** — where the data is dense, Ex\_03 gives 1.8 mm
- **many dimensions** — correlations nobody can write down
- **speed** — microseconds an evaluation, once trained
- **inverse problems** — a parameter you cannot measure directly

## THE COMMON SHAPE

Complicated function, simple objective — that is the shape.

The method supplies the function. You must supply the objective.

Which means the engineering judgement moves entirely into the loss. Choose it badly and you will get a model that optimises exactly what you asked for and nothing you wanted.

In Part 2 the loss stops being a formality and becomes the design.

# When Not To Use It

**When not to use it** — the three conditions that should stop you: data too thin to support the parameters, a guarantee that an error measured on the data you happened to test cannot give, and physics that is already known and cheap to solve.

## YOUR DATA IS THIN

Fit a curve instead. A model with a handful of physical parameters is defensible on the data you have; one with thousands is not, however good the training error looks.

## YOU NEED A GUARANTEE

Certification wants bounds. A trained network gives an empirical error on the data you happened to test — a different kind of object.

## THE PHYSICS IS KNOWN AND CHEAP

If a solver runs in a second, use the solver. A network that took six hours and is wrong unpredictably is not an improvement.

## ASK IN THIS ORDER

- |              |                                                 |
|--------------|-------------------------------------------------|
| <b>first</b> | can I write the equation down?                  |
| <b>then</b>  | is solving it fast enough?                      |
| <b>then</b>  | do I have data in the hundreds or the millions? |
| <b>then</b>  | who has to believe the answer?                  |
| <b>last</b>  | and only then, which architecture?              |

*Almost every failed machine-learning project in engineering skipped straight to the last question. The ordering is the content of this slide.*

# Data Requirements

**Data requirements** — how many examples a model needs, which has no answer on its own. It depends on how much physics you write into the model: with none, thousands and still nothing trustworthy outside them; with all of it, no measurements at all. How much data is the wrong question asked first.

## THE SAME PROBLEM, THREE WAYS

- **no physics** — thousands of examples, and still blind outside them
- **some physics** — 101 noisy points, 0.59 mm outside
- **all physics** — the equation and one initial condition

## ORDERS OF MAGNITUDE

|                                |                            |
|--------------------------------|----------------------------|
| <b>a thesis test campaign</b>  | $10^2$ runs                |
| <b>a year of plant SCADA</b>   | $10^6$ samples, few events |
| <b>ImageNet</b>                | $10^6$ labelled images     |
| <b>a modern language model</b> | $10^{12}$ tokens           |
| <b>a PINN, in Part 2</b>       | zero labelled outputs      |

*The last row is not a trick. In Part 2 you will train a network that has never been shown a correct answer, and it will produce one.*

# Advanced Topics

## RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

| topic                      | what it adds                       | read            | where it makes a difference    |
|----------------------------|------------------------------------|-----------------|--------------------------------|
| symbolic AI                | logic and search, with no learning | UDL ch. 1 notes | planning, scheduling, proof    |
| classical machine learning | SVM, trees and k-NN on small data  | UDL ch. 1 notes | hundreds of rows, tabular data |
| generative models          | a model that produces new data     | UDL ch. 14 – 18 | design search, synthetic data  |
| Bayesian methods           | a distribution over the answer     | UDL ch. 1 notes | uncertainty you have to report |
| reproducibility            | seeds, splits and honest reporting | UDL ch. 1 notes | every claim you will read      |

### WHY THEY ARE LEFT OUT

Each is a field beside this one rather than a step inside it, and most pay off where the data is small or the answer has to carry a probability. This course follows the one method that runs through all twelve lectures.

# Exercise 3

## WHAT THE NOTEBOOK DOES

One notebook, one problem: a mass on a spring, measured for five seconds by a sensor that is noisy and was never zeroed.

You build the same system five ways, from the analytic solution through a fitted regression and a plain network to a network with the equation of motion in its loss, and you compare them on data they never saw.

The network agrees with the measurements to 0.62 mm. The measurements themselves are wrong by 1.80 mm, so it fitted the noise, and outside the window it is fifty times worse than the model that was told the physics.

The fifth model uses no measurements at all. The equation and one initial condition are enough.

## THE QUESTION IT ANSWERS

**Ex\_03**                      which model would you sign?

**models 1 – 3**              what does fitting noise cost?

**models 4 – 5**              what does the equation buy?

## REPORT QUESTION, TO CONCLUDE

All five models fit the data. Which one would you put in a design report, and what number in your own results defends that choice? The marks are for the reasoning, not for the error.

# Key Takeaways

## DEEP LEARNING

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| <b>what it is</b>  | many layers, fitted end to end by gradient descent      |
| <b>function</b>    | inputs in, a prediction out, parameters fitted to data  |
| <b>train, tune</b> | a function, a loss, an optimiser: the three ingredients |
| <b>vs physics</b>  | faster, and blind outside the data it saw               |
| <b>used for</b>    | interpolation, many dimensions, speed, inverse problems |

## THE MODELLING SPECTRUM

|                     |                                                    |
|---------------------|----------------------------------------------------|
| <b>what it is</b>   | the line from pure physics to pure data            |
| <b>function</b>     | places any model, including the ones you built     |
| <b>train, tune</b>  | the physics you write in decides the data you need |
| <b>vs pure data</b> | Ex_03: 0.59 mm outside the window against 30.26    |
| <b>used for</b>     | deciding whether to use a network at all           |

# Questions

## 1 – 2 · DEEP LEARNING

### 1 What is artificial intelligence, and how are machine learning and deep learning nested inside it?

AI is any program that does a task we would call intelligent. Machine learning is the part that fits its behaviour to data instead of following written rules. Deep learning is machine learning with a network of many layers, which finds its own features in the raw input. *UDL ch. 1*

### 2 What does a machine-learning model compute, and what are its three ingredients?

A function from inputs to a prediction, with parameters fitted to data. The ingredients are the function, a loss that measures how wrong the prediction is, and an optimiser that changes the parameters to lower the loss. *UDL ch. 2*

## 3 – 4 · THE MODELLING SPECTRUM

### 3 What is the modelling spectrum, and what does it decide?

The line from pure physics to pure data, on which every model has a place. The more physics is written into a model, the less data it needs and the further it holds outside that data. In Ex\_03 the fit with the physics in it missed by 0.59 mm outside the window, the pure data fit by 30.26 mm. *UDL ch. 1.4; Ex\_03*

### 4 When does a trained network earn its place, and when should you not use one?

It earns it for interpolation inside dense data, for many input dimensions, for speed and for inverse problems. Do not use one when the data is too thin for its parameters, when a guarantee is required, or when the physics is known and cheap to solve. *UDL ch. 1.4*

NEXT

## L3.2 — What the Field Has Built

---

Today was what the field is and how it got here. Next time is what it has actually produced — across vision, language, control, and then across science and engineering, which is where this course is going.

### BEFORE THE NEXT SESSION

|                    |                                      |
|--------------------|--------------------------------------|
| <b>read</b>        | UDL chapter 1 — about twenty minutes |
| <b>run</b>         | Ex_03, to check your setup           |
| <b>bring</b>       | one model from your own work         |
| <b>think about</b> | where on the spectrum it sits        |

*Bring the model. L3.2 ends by placing the room's own examples on the spectrum, and a lecture theatre full of engineers produces better cases than any set I could invent — but only if you arrive having thought about one.*